# RL Homework 6

November 27, 2025

## 1. Function Approximation

### (a) Tabular methods as a special case

In linear function approximation, we approximate the value function using the dot product of a weight vector $\mathbf{w}$ and a feature vector $\mathbf{x}(s)$:

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s) = \sum_{i=1}^{d} w_i x_i(s)$$

To make this behave like a tabular method, we can define the feature vector $\mathbf{x}(s)$ as a one-hot encoded vector. If we have a discrete state space with $N$ states, the feature vector has a size of $N$. For a specific state $s_k$, the feature vector would look like this:

$$\mathbf{x}(s_k) = [0, 0, \ldots, 1, \ldots, 0]^T$$

Where the entry at index K is 1. Using this into the linear equation, we get:

$$\hat{v}(s_k, \mathbf{w}) = w_k \cdot 1 + \sum_{j \neq k} w_j \cdot 0 = w_k$$

This means the approximated value for state $s_k$ is just the weight $w_k$. Since every state is representing their own weights, this is same as using a table to store the values.

### (b) Polynomial features dimensionality

We are looking at a state space with $k$ dimensions, $s = (s_1, ..., s_k)$. The basis functions are products of these state variables raised to powers of different values:

$$x_i(s) = \prod_{j=1}^{k} s_j^{c_{i,j}}$$

Counting how many different combinations of exponents are there. For all $k$ dimensions of the state, since the choice of exponent for each dimension is independent, we multiply the number of choices together:

$$\text{Total Features} = \underbrace{(n+1) \cdot (n+1) \cdot ... \cdot (n+1)}_{k \text{ times}} = (n+1)^k$$

This is why the feature space has $(n+1)^k$ dimensions.

## 2. Feature Designing

We have chosen a feature vector with 7 components.

Let $\mathbf{x}(s) = [x_1, x_2, x_3, x_4, x_5, x_6, x_7]^T$:

- $x_1$: *Mario's X-coordinate.* This represents Mario's horizontal position on the screen (normalized between 0 and 1), which tells the agent how far right it has moved.
- $x_2$: *Mario's Y-coordinate.* This represents the vertical position of Mario on screen (again normalized between 0 and 1). It helps the agent know if it is on the ground or in the jumping action.
- $x_3$: *Closeness to nearest enemy X-coordinate.* This is calculated as $(x_{enemy} - x_{mario})$.
- $x_4$: *Closeness to nearest enemy Y-coordinate.* This is $(y_{enemy} - y_{mario})$.
- $x_5$: *Closeness to nearest object X-coordinate.* When an object is blocking the path ahead this is needed to learn to jump.
- $x_6$: *Closeness to nearest object Y-coordinate.* When an object is blocking the path ahead this is needed to learn to jump.
- $x_7$: *Jump state* – Indicates the agent's vertical state (e.g., 1 if jumping, 0 otherwise).
- $x_8$: *Item state* – Represents the current item status (e.g., binary indicator for possession or categorical ID of the item).
- $x_9$: *Life numbering* – Represents the remaining number of lives or health points of the agent.
- $x_{10}$: *X-axis speed* – Represents the velocity of the agent along the horizontal axis.
- $x_{11}$: *Bias term* – A constant value set to 1, allowing the linear function to learn an intercept.

## 3. Value function fitting

### (a) Plot a trajectory of the system

We visualized the trajectory of the system using the given PD policy. The plots below show the state transitions $(\cos\theta, \sin\theta, \dot{\theta})$, the actions (torque), and the rewards over time. The system tries but is not perfectly stabilized in the upright position.

To see clearly we visualized in two step ranges. 50000 and 500.

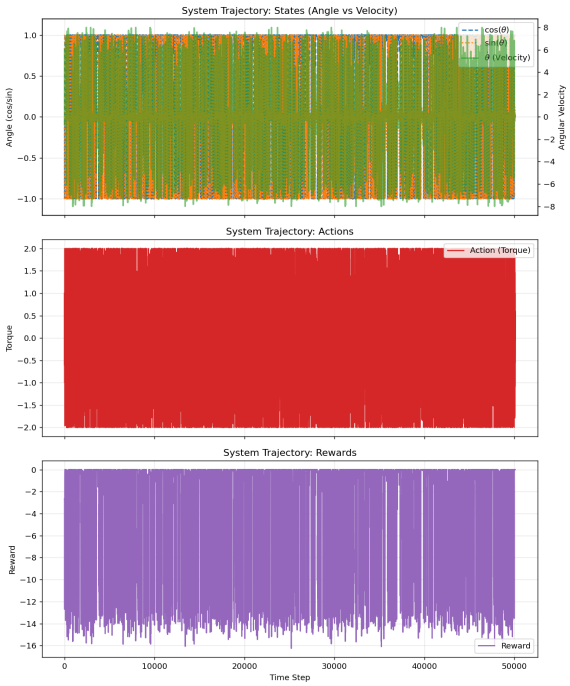


Figure 1: Timestep till 50000

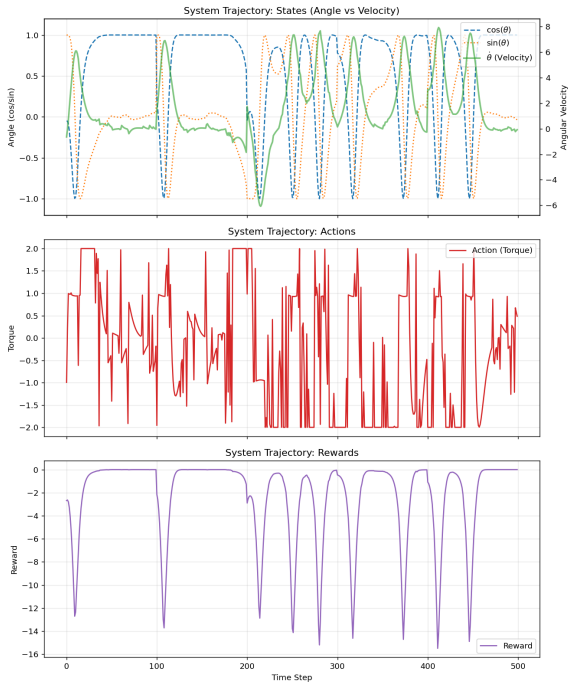


Figure 2: Timestep till 500

**(b) Use a neural network function approximator, to fit the value function. Use an appropriate target (TD update) for the value fitting with Y = 0.95.Monitor the learning progress.**

Starting training...

Iter 0, Loss: 19.15809
Iter 200, Loss: 15.41148
Iter 400, Loss: 17.03938
Iter 600, Loss: 17.47271
Iter 800, Loss: 15.14911
Iter 1000, Loss: 13.99604
Iter 1200, Loss: 12.38872
Iter 1400, Loss: 10.40890
Iter 1600, Loss: 10.37769
Iter 1800, Loss: 8.25482

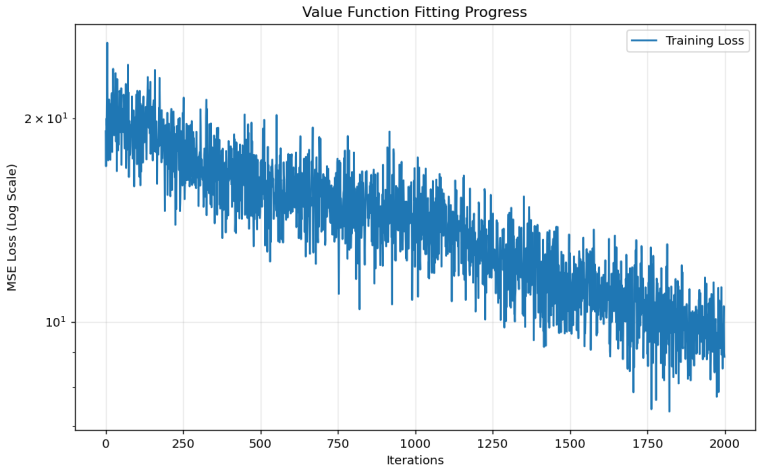


Figure 3: Learning Curve

## (c) Plot the value function before and after learning

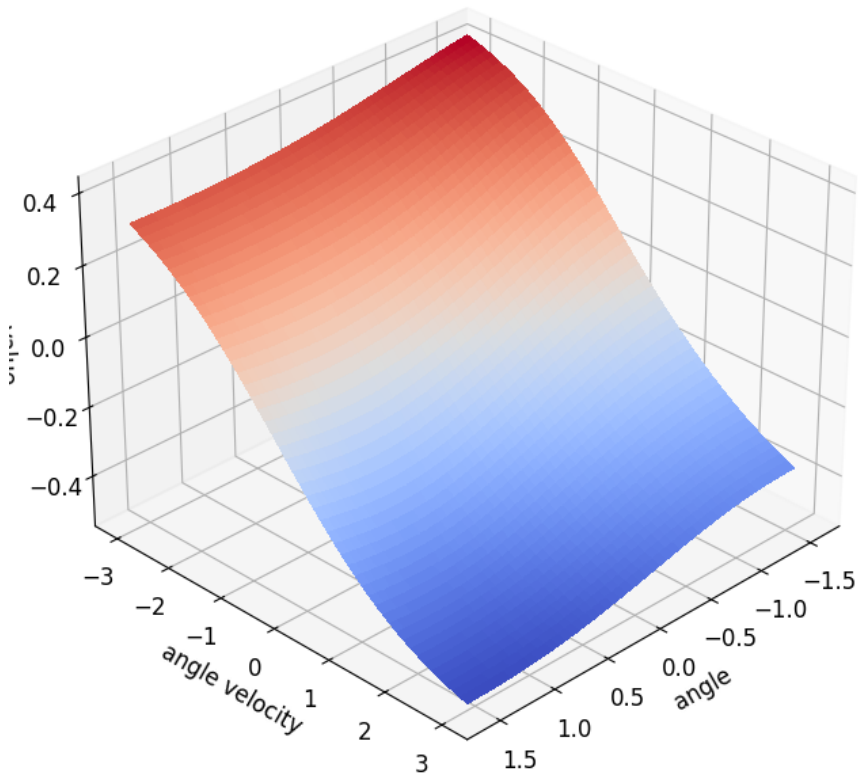


Figure 4: Value Function Before Training

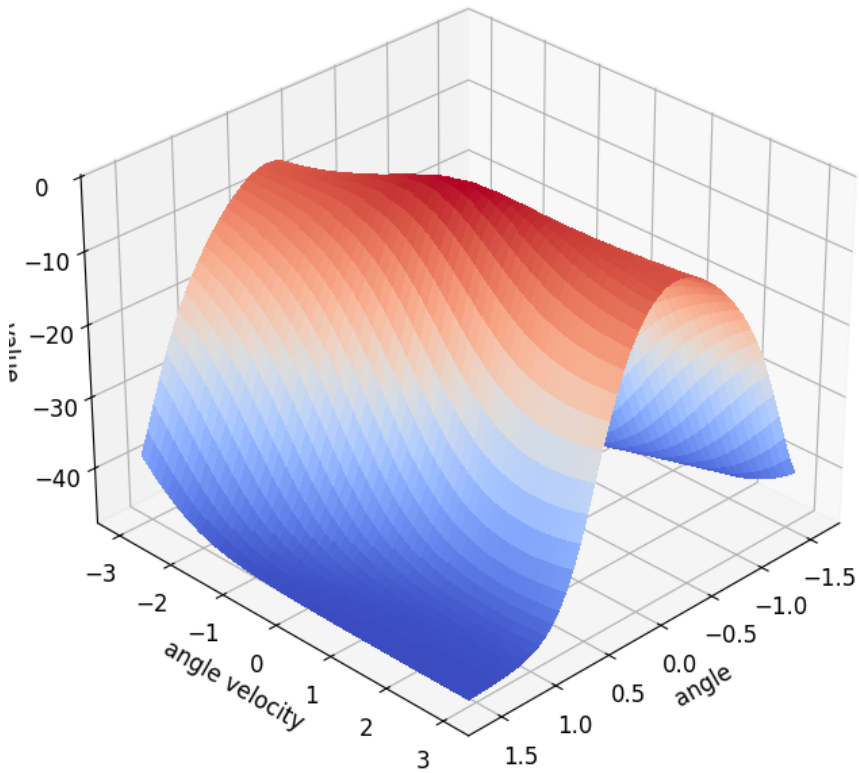


Figure 5: Value Function After Training

### (d) Give an intuitive interpretation of the plotted value function.

The learned value function landscape provides a clear map of the "desirability" of each state in the Pendulum environment. We can observe three key characteristics:

1. **Global Maximum at the Goal State $(0, 0)$:**
   The highest value (closest to 0, represented in red) is located exactly at the center where the angle is 0 (upright) and the angular velocity is 0. This is intuitive because the goal of the agent is to balance the pole upright, which yields the maximum reward defined in the environment.

2. **steep Gradient (The "Bowl" Shape):**
   As the state moves away from the center—either the pole falls (angle deviates) or spins too fast (high velocity)—the value drops sharply. This steep gradient acts as a guide for the policy. To maximize future rewards, the agent must "climb" this slope towards the peak, which physically corresponds to stabilizing the pendulum.

3. **Spiral Ridges (Swing-up Trajectories):**
   Interestingly, the value function is not a simple cone. We can observe curved "ridges" or spiral patterns extending from the center. These ridges represent optimal *swing-up trajectories*. Even if the pendulum is currently at the bottom (Angle $\approx \pm\pi$), states with the correct velocity to swing up towards the top are valued much higher than stationary states at the bottom. This structure indicates that the network has learned the dynamics required to gather momentum and swing the pole up.

### (e) Compare the learning curve and the resulting value function when using Y = 0.5. What happens?

When the discount factor is reduced to $\gamma = 0.5$, the agent becomes significantly more myopic, prioritizing immediate rewards over long-term stability. As a result, the magnitude of the learned value function decreases considerably compared to the $\gamma = 0.95$ case, since future rewards contribute less to the cumulative sum. Visually, the value function surface becomes much flatter overall, with a sharp peak only in the immediate vicinity of the goal state (0,0). This indicates the agent fails to recognize the value of "strategic" states far from the goal that are crucial for the long-term swing-up maneuver, effectively only caring about being upright in the very near future.

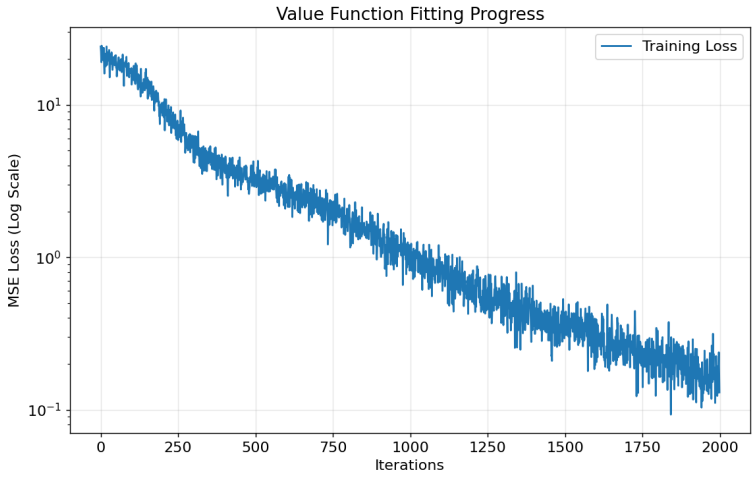


Figure 6: Learning Curve with Gamma 0.5

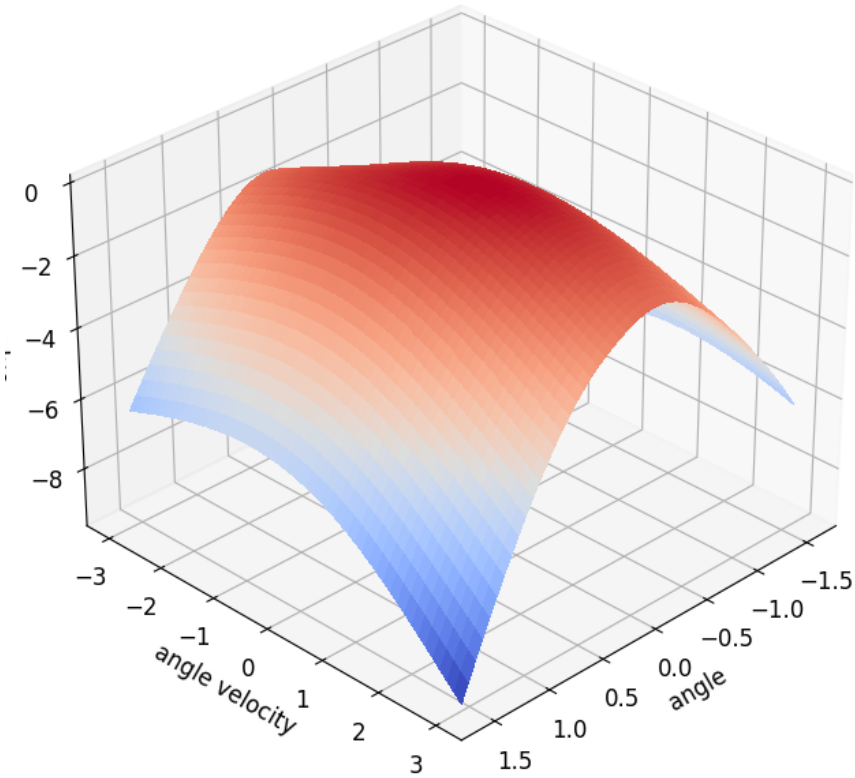


Figure 7: Value Function with Gamma 0.5